

Rapport de Projet

Overcooked Deluxe - Jeu de Cuisine Multi-Recettes

Groupe 1

Membres du groupe :

- ALJIBBAOUI Tala
- AMMARI Reda
- CALDERON Matéo
- DIEVART Colin

Table des matières

1	Introduction	3
2	Technologies Utilisées	3
2.1	Langage de Programmation	3
2.2	Bibliothèques Principales	3
3	Architecture du Projet	3
3.1	Structure des Fichiers	3
3.2	Pattern MVC	3
4	Fonctionnalités Implémentées	3
4.1	Système de Recettes	3
4.2	Stations de Travail	4
4.3	Système de Commandes	4
5	Intelligence Artificielle - Agent Autonome	4
5.1	Justification : Un Véritable Agent selon Wooldridge	4
5.1.1	Définition d'Agent	4
5.1.2	Autonomie	4
5.1.3	Pro-activité (Goal-directed behavior)	4
5.1.4	Capacité Sociale	4
5.2	Architecture Formelle de l'Agent	4
5.2.1	Environnement	4
5.2.2	Architecture "Agent with State"	5
5.3	Répertoire d'Actions	5
5.4	Exemples de Runs	5
5.5	Fonction Utilitaire	6
6	Fonctions Essentielles	6
6.1	Perception de l'Environnement	6
6.2	Sélection d'Action	6
6.3	Planification de Recette	7
6.4	Gestion de l'Assemblage	7
6.5	Système de Cuisson	8
7	Difficultés Rencontrées	8
7.1	Synchronisation des Actions	8
7.2	Gestion des Commandes Expirées	8
7.3	Réutilisation des Plats	8
7.4	Collision des Animations	9
7.5	Gestion de la Cuisson	9
8	Améliorations Possibles	9

1 Introduction

Ce projet consiste en la réalisation d'un jeu inspiré d'Overcooked, développé en Python avec la bibliothèque Pygame. Le jeu simule la gestion d'une cuisine où un chef (contrôlé par une IA) doit préparer et livrer des commandes de burgers et pizzas dans un temps limité.

L'objectif principal est de mettre en œuvre une architecture de jeu basée sur le pattern Modèle-Vue-Contrôleur (MVC) et d'implémenter un agent autonome intelligent capable de gérer les tâches de cuisine de manière efficace.

2 Technologies Utilisées

2.1 Langage de Programmation

Le projet a été entièrement développé en **Python 3**, un langage de programmation de haut niveau particulièrement adapté pour :

- Le développement rapide de prototypes
- La programmation orientée objet
- L'implémentation d'algorithmes d'intelligence artificielle

2.2 Bibliothèques Principales

Bibliothèque	Utilisation
pygame	Gestion de l'affichage graphique, des événements et du rendu 2D
dataclasses	Simplification de la création de classes pour les modèles de données
enum	Définition d'énumérations pour les types d'items et de stations
time	Gestion du temps (cuisson, commandes, animations)
typing	Annotations de type pour améliorer la lisibilité et la maintenance

TABLE 1 – Bibliothèques utilisées dans le projet

3 Architecture du Projet

Le projet suit une architecture MVC (Modèle-Vue-Contrôleur) organisée en modules distincts :

3.1 Structure des Fichiers

- `game_model.py` : Modèle du jeu (logique métier, état du jeu)
- `game.view.py` : Vue (rendu graphique, interface utilisateur)
- `game.controller.py` : Contrôleur principal (boucle de jeu)
- `bot_controller.py` : Agent IA autonome

3.2 Pattern MVC

Modèle : Gère l'état du jeu (positions des joueurs, items, stations, commandes, score).

Vue : Responsable de l'affichage (cuisine, chef, clients, UI).

Contrôleur : Coordonne les interactions entre le modèle et la vue, gère les entrées utilisateur et met à jour le jeu.

4 Fonctionnalités Implémentées

4.1 Système de Recettes

Le jeu supporte plusieurs recettes avec des ingrédients et préparations spécifiques :

- **Burger** : Pain + Steak cuit + Tomate coupée + Salade coupée
- **Pizza** : Pain + Tomate coupée + Fromage (puis on la met au four)
- **Salade** : Salade + Tomate

4.2 Stations de Travail

- **Spawn** : Points de génération des ingrédients
- **Cutting Board** : Découpe des légumes
- **Stove** : Cuisson de la viande
- **Assembly** : Assemblage des plats
- **Delivery** : Livraison aux clients

4.3 Système de Commandes

- Génération aléatoire de commandes toutes les 15-30 secondes
- Maximum de 3 commandes actives simultanément
- Temps limité par commande (60 secondes)
- Bonus de score selon le temps de livraison
- Pénalités pour les commandes expirées ou la nourriture trop cuite

5 Intelligence Artificielle - Agent Autonome

5.1 Justification : Un Véritable Agent selon Wooldridge

Notre système implémente un agent autonome conforme à la définition formelle de Wooldridge. Analysons point par point :

5.1.1 Définition d'Agent

Selon Wooldridge, *"Un agent est un système informatique capable d'action autonome dans un environnement afin d'atteindre ses objectifs de conception"*. Notre agent respecte cette définition :

- **Système informatique** : Implémenté en Python (classe `AIBot`)
- **Action autonome** : Prend des décisions sans intervention humaine constante
- **Environnement** : La cuisine avec ses stations, ingrédients et commandes
- **Objectifs** : Compléter les commandes pour maximiser le score

5.1.2 Autonomie

L'agent fait preuve d'autonomie en :

- Décidant quelles commandes traiter en priorité
- Planifiant la séquence d'actions nécessaires
- S'adaptant aux changements (commandes expirées, plats disponibles)
- Contrôlant son état interne sans supervision externe

5.1.3 Pro-activité (Goal-directed behavior)

L'agent n'est pas simplement réactif, il est **pro-actif** :

- Il génère des plans pour atteindre ses objectifs (fonction `plan_recipe`)
- Il prend des initiatives (sélectionner la commande la plus urgente)
- Il anticipe les besoins (commencer la cuisson à l'avance)
- Il ne se contente pas de réagir aux stimuli

5.1.4 Capacité Sociale

L'agent interagit avec son environnement :

- Communication avec les stations de travail (spawn, cutting board, stove, assembly, delivery)
- Coordination implicite si plusieurs agents étaient présents
- Adapte son comportement selon l'état des autres éléments

5.2 Architecture Formelle de l'Agent

5.2.1 Environnement

L'environnement E de l'agent est défini par :

- **États** : Positions des joueurs, états des stations (items présents, cuisson en cours), commandes actives, score

- **Accessible** : Partiellement - l'agent voit tout instantanément mais la cuisson est un processus temporel dont l'issue peut varier
- **Non-déterministe** : Les commandes arrivent aléatoirement, les délais varient, risque de sur-cuisson
- **Dynamique** : Les commandes évoluent, le temps s'écoule, les états de cuisson changent indépendamment de l'agent

5.2.2 Architecture "Agent with State"

Notre agent implémente l'architecture "**Agents with State**" de Wooldridge :

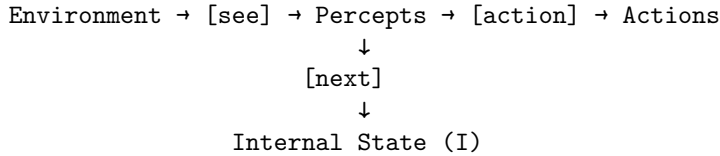


FIGURE 1 – Architecture de l'agent avec état interne

Fonction see : `perceive(m: GameModel) → Dict`

Transforme l'état de l'environnement en percepts (position, items tenus, commandes actives, états des stations).

État interne I : Ensemble des états internes possibles

- `AgentState.IDLE` : En attente d'une tâche
- `AgentState.EXECUTING_RECIPE` : Préparation d'une recette
- `AgentState.DELIVERING` : Livraison d'un plat
- Variables : `current_recipe`, `current_order`, `queue` (plan d'actions)

Fonction action : `action(percepts: Dict)`

Sélectionne l'action à effectuer en fonction de l'état interne.

Fonction next : `_update_internal_state(percepts: Dict)`

Met à jour l'état interne basé sur l'état actuel et les nouveaux percepts.

5.3 Répertoire d'Actions

L'agent dispose d'un ensemble d'actions Ac :

- `GO_TO(station)` : Se déplacer vers une station
 - *Pré-condition* : Station accessible
 - *Post-condition* : Position = position de la station
- `INTERACT(station)` : Interagir avec une station
 - *Pré-condition* : Proximité de la station
 - *Post-condition* : Item pris/déposé selon contexte
- `CHOP(station)` : Découper un ingrédient
 - *Pré-condition* : Item découppable sur planche
 - *Post-condition* : Item.chopped = True
- `WAIT(duration)` : Attendre
 - *Pré-condition* : Aucune
 - *Post-condition* : Temps écoulé

5.4 Exemples de Runs

Run 1 - Préparation d'un Burger :

1. État : IDLE, aucun item tenu, commande burger apparaît
2. Percept : Nouvelle commande détectée
3. Action : Sélectionner commande, passer en EXECUTING_RECIPE
4. État : Planning de la recette burger
5. Actions séquentielles :
 - `GO_TO(spawn_bread) → INTERACT → GO_TO(assembly) → INTERACT`

- GO_TO(spawn_patty) → INTERACT → GO_TO(stove) → INTERACT
- WAIT(3s) [cuisson]
- GO_TO(stove) → INTERACT → GO_TO(assembly) → INTERACT
- GO_TO(spawn_tomato) → INTERACT → GO_TO(board) → INTERACT → CHOP
→ INTERACT → GO_TO(assembly) → INTERACT

Idem pour lettuce

6. État : DELIVERING, burger assemblé
7. Action : GO_TO(delivery) → INTERACT
8. Percept : Score augmenté, commande complétée
9. État : Retour IDLE

Run 2 - Adaptation à une commande expirée :

1. État : EXECUTING_RECIPES, préparation en cours
2. Percept : current_order_id not in active_order_ids
3. Action : _abandon_current_task()
4. État : IDLE, queue vidée
5. Percept : Nouvelle commande disponible
6. Action : Sélectionner nouvelle commande

5.5 Fonction Utilitaire

L'agent utilise implicitement une fonction utilitaire basée sur le score :

- **Maximisation** : Score = récompenses - pénalités
- **Récompenses** : +15\$ par livraison + bonus temps
- **Pénalités** : -20\$ commande expirée, -10\$ plat trop cuit
- **Stratégie** : Prioriser commandes urgentes (temps restant faible)

6 Fonctions Essentielles

6.1 Perception de l'Environnement

```

1 def perceive(self, m: GameModel) -> Dict:
2     """
3     Fonction 'see' : perçoit l'état de l'environnement
4     Retourne un dictionnaire de percepts
5     """
6     p = self._p(m)
7
8     percepts = {
9         'player_position': (p.x, p.y),
10        'held_item': p.held_item,
11        'active_orders': m.orders.copy(),
12        'active_order_ids': [o.id for o in m.orders],
13        'score': m.score,
14        'stations_state': self._perceive_stations(m),
15        'assembly_state': self._perceive_assembly(m),
16    }
17
18    return percepts

```

Utilité : Cette fonction permet à l'agent de collecter toutes les informations nécessaires sur l'état actuel du jeu sans accéder directement au modèle complet.

6.2 Sélection d'Action

```

1 def action(self, percepts: Dict) -> None:
2     """
3     Fonction 'action' : décide de l'action à prendre
4     basée sur l'état interne et les percepts
5     """

```

```

6      # Vérifier si la commande existe encore
7      if self.current_order_id is not None:
8          if self.current_order_id not in percepts['active_order_ids']:
9              self._abandon_current_task(percepts)
10             return
11
12     # Mettre à jour l'état interne
13     self._update_internal_state(percepts)
14
15     # Sélectionner la prochaine action
16     if self.internal_state == AgentState.IDLE:
17         self._select_order(percepts)
18     elif self.internal_state == AgentState.EXECUTING_RECIPE:
19         if not self.queue:
20             self._plan_recipe(percepts)

```

Utilité : Cœur du système de décision de l'agent. Cette fonction détermine quelle action entreprendre en fonction de l'état actuel et des informations perçues.

6.3 Planification de Recette

```

1  def _plan_ingredient(self, m: GameModel, ingredient: ItemType,
2                      needs_chopping: bool):
3      """Planifie la préparation d'un ingrédient spécifique"""
4      p = self._p(m)
5      a = self._assembly(m)
6
7      # Cas spécial: RAW_PATTY doit être cuit
8      needs_cooking = (ingredient == ItemType.RAW_PATTY)
9      cooked_version = ItemType.COOKED_PATTY if needs_cooking else ingredient
10
11     # Si on tient déjà l'ingrédient préparé
12     if p.held_item and p.held_item.item_type == cooked_version:
13         if needs_chopping and not p.held_item.chopped:
14             # Doit être coupé
15             board = self._free_board(m)
16             if board:
17                 self._push_with_gap(Step.GO_TO, board)
18                 self._push_with_gap(Step.INTERACT, board)
19                 self._push_with_gap(Step.CHOP, board)
20             # ... suite du plan

```

Utilité : Génère une séquence d'actions pour préparer un ingrédient spécifique selon ses besoins (découpe, cuisson). Gère les cas particuliers comme la cuisson de la viande.

6.4 Gestion de l'Assemblage

```

1  def _check_recipe_completion(self, station: Station):
2      """Vérifie si les ingrédients forment un plat complet"""
3      types = {item.item_type for item in station.contents}
4
5      # Vérifier si un item est overcook
6      has_overcooked = any(getattr(item, 'overcooked', False)
7                           for item in station.contents)
8
9      # Burger: pain + steak cuit + tomate coupée + salade coupée
10     if (ItemType.BREAD in types and
11         ItemType.COOKED_PATTY in types and
12         any(i.item_type == ItemType.TOMATO and i.chopped
13             for i in station.contents) and
14         any(i.item_type == ItemType.LETTUCE and i.chopped
15             for i in station.contents)):
16         burger = Item(ItemType.BURGER)
17         burger.overcooked = has_overcooked
18         station.item = burger

```

```
19 station.contents.clear()
```

Utilité : Vérifie automatiquement si tous les ingrédients nécessaires sont présents sur la station d'assemblage et crée le plat final. Gère aussi l'état "overcook".

6.5 Système de Cuisson

```
1 def update(self, delta_time: float):
2     """Met à jour le modèle de jeu"""
3     current_time = time.time()
4
5     # Mise à jour des stations (cuisson et sur-cuisson)
6     for station in self.stations:
7         if (station.station_type == StationType.STOVE and
8             station.item and
9             station.item.item_type == ItemType.RAW_PATTY and
10            station.cooking_start_time > 0):
11
12            cooking_time = current_time - station.cooking_start_time
13
14            # Cuit parfaitement
15            if (cooking_time >= station.cooking_duration and
16                cooking_time < station.overcook_duration):
17                if station.item.item_type != ItemType.COOKED_PATTY:
18                    station.item = Item(ItemType.COOKED_PATTY)
19                    print("    Steak parfaitement cuit!")
20
21            # Trop cuit / brûlé
22            elif cooking_time >= station.overcook_duration:
23                if station.item.item_type != ItemType.BURNT_PATTY:
24                    station.item = Item(ItemType.BURNT_PATTY,
25                                         overcooked=True)
26                    print("    Steak brûlé! (Overcooked)")
```

Utilité : Gère la progression de la cuisson en temps réel avec transitions d'état (cru → cuit → brûlé). Système critique pour le gameplay.

7 Difficultés Rencontrées

7.1 Synchronisation des Actions

Problème : L'agent effectuait les actions trop rapidement, causant des bugs (interactions multiples, déplacements saccadés).

Solution : Implémentation d'un système de cooldown avec `.step_gap` et `.gap_until` pour espacer les actions.

7.2 Gestion des Commandes Expirées

Problème : L'agent continuait à travailler sur des commandes déjà expirées ou complétées par erreur.

Solution : Ajout d'un système de tracking par ID unique (`current_order_id`) et vérification à chaque cycle avec abandon automatique si la commande n'existe plus.

7.3 Réutilisation des Plats

Problème : Les plats assemblés pour des commandes expirées restaient inutilisés sur la station d'assemblage.

Solution : L'agent vérifie maintenant si un plat terminé est disponible sur l'assemblage et peut le réutiliser pour une nouvelle commande, sauf s'il est "overcook".

7.4 Collision des Animations

Problème : Les clients se superposaient visuellement dans l'interface.

Solution : Implémentation d'un système de positionnement dynamique avec espacement calculé en fonction du nombre de commandes actives.

7.5 Gestion de la Cuisson

Problème : Déterminer le bon moment pour retirer la viande du feu (ni trop tôt, ni trop tard).

Solution : L'agent calcule le temps de cuisson restant et attend dynamiquement près du fourneau, avec vérification de l'état "overcook".

8 Améliorations Possibles

- **Multi-agents** : Ajouter plusieurs chefs qui coopèrent
- **Pathfinding avancé** : Algorithme A* pour éviter les obstacles
- **Apprentissage par renforcement** : Optimiser les stratégies de l'IA
- **Niveaux de difficulté** : Cuisine plus grande, plus d'ingrédients
- **Mode multijoueur** : Permettre à plusieurs joueurs humains de jouer
- **Nouvelles recettes** : Salades, desserts, boissons

9 Conclusion

Ce projet a permis de mettre en pratique plusieurs concepts importants :

- Architecture logicielle (MVC)
- Intelligence artificielle (agent autonome)
- Programmation orientée objet
- Gestion d'événements et d'états
- Développement de jeux vidéo

Le résultat est un jeu fonctionnel avec une IA capable de gérer efficacement les commandes, de planifier les actions et de s'adapter aux changements de l'environnement. Les difficultés rencontrées ont permis d'approfondir la compréhension des systèmes temps réel et de la coordination d'actions complexes.